

Lezione 3

Digital systems & VHDL

Stefan Cristi Zugravel
19/01/2026

Digital systems

analog signals :

- *continuous in both time and amplitude*
- *usually a voltage $v(t)$ or a current $i(t)$ as a function of time*
- *reach of information (e.g. frequency spectrum, FFT)*

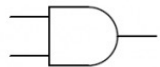
digital signals :

- *usually continuous in time but discrete in amplitude*
- *only two possible values e.g. high/low voltage levels, true/false, on/ff, closed/open etc.*
- *less information, but more robust against noise*
- *can be either asynchronous signals or synchronous signals*

Classification of digital circuits

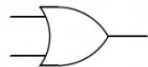
Digital circuits are classified as :

- combinational circuits
- sequential circuits
 - asynchronous
 - synchronous



AND

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1



OR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1



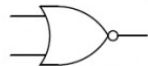
XOR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0



NAND

A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0



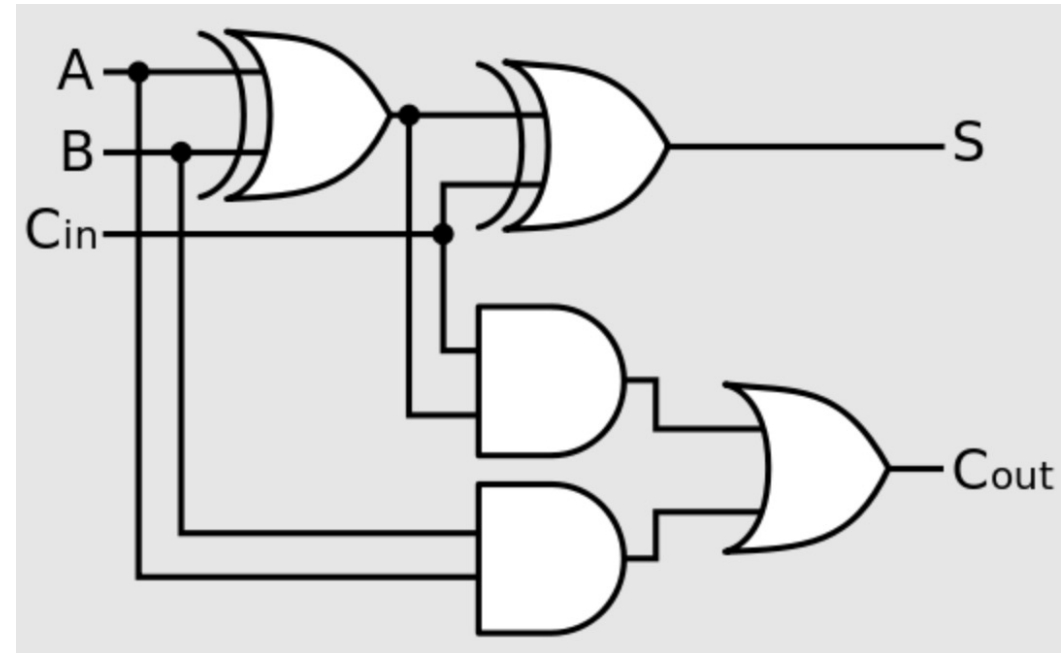
NOR

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0

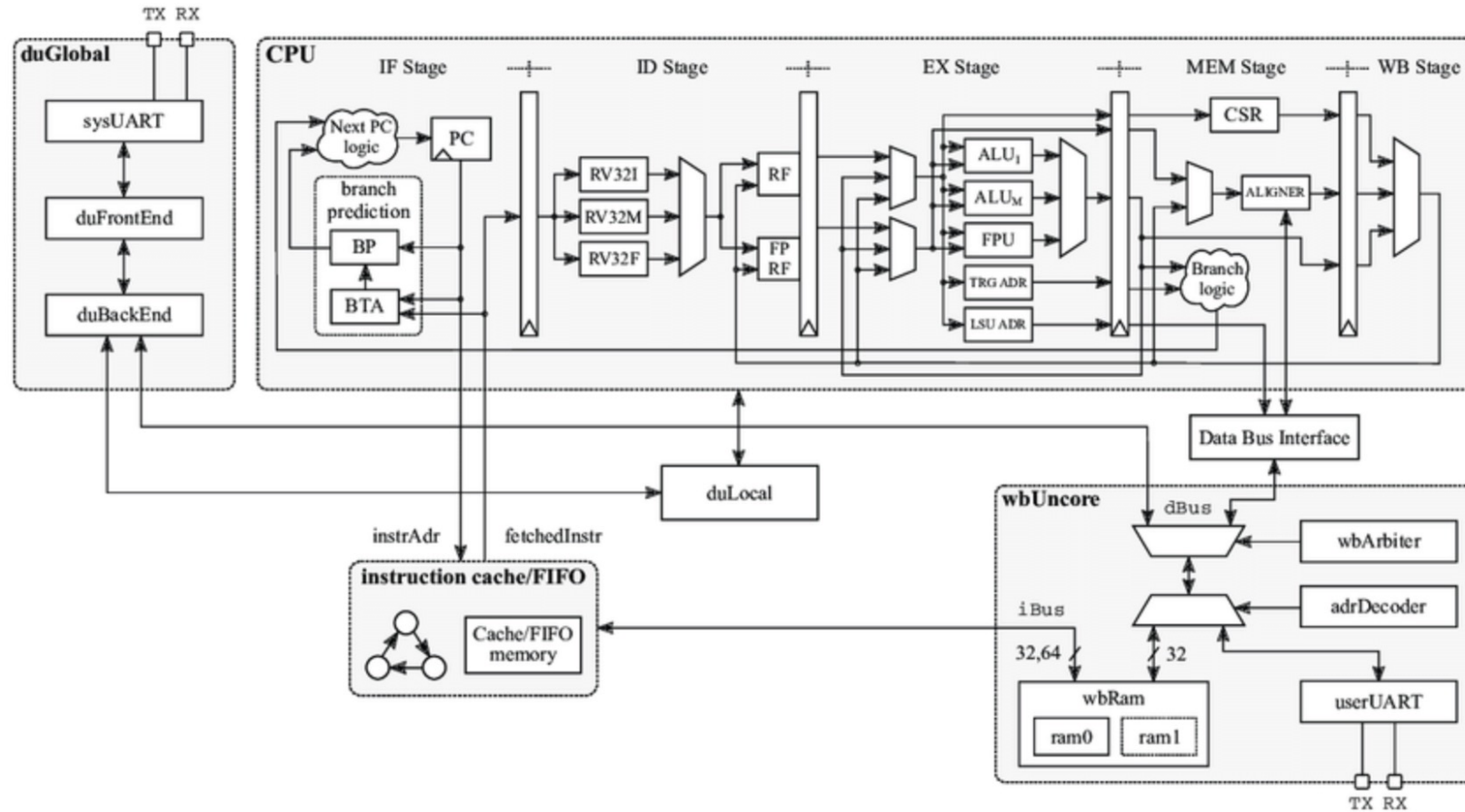


XNOR

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	1

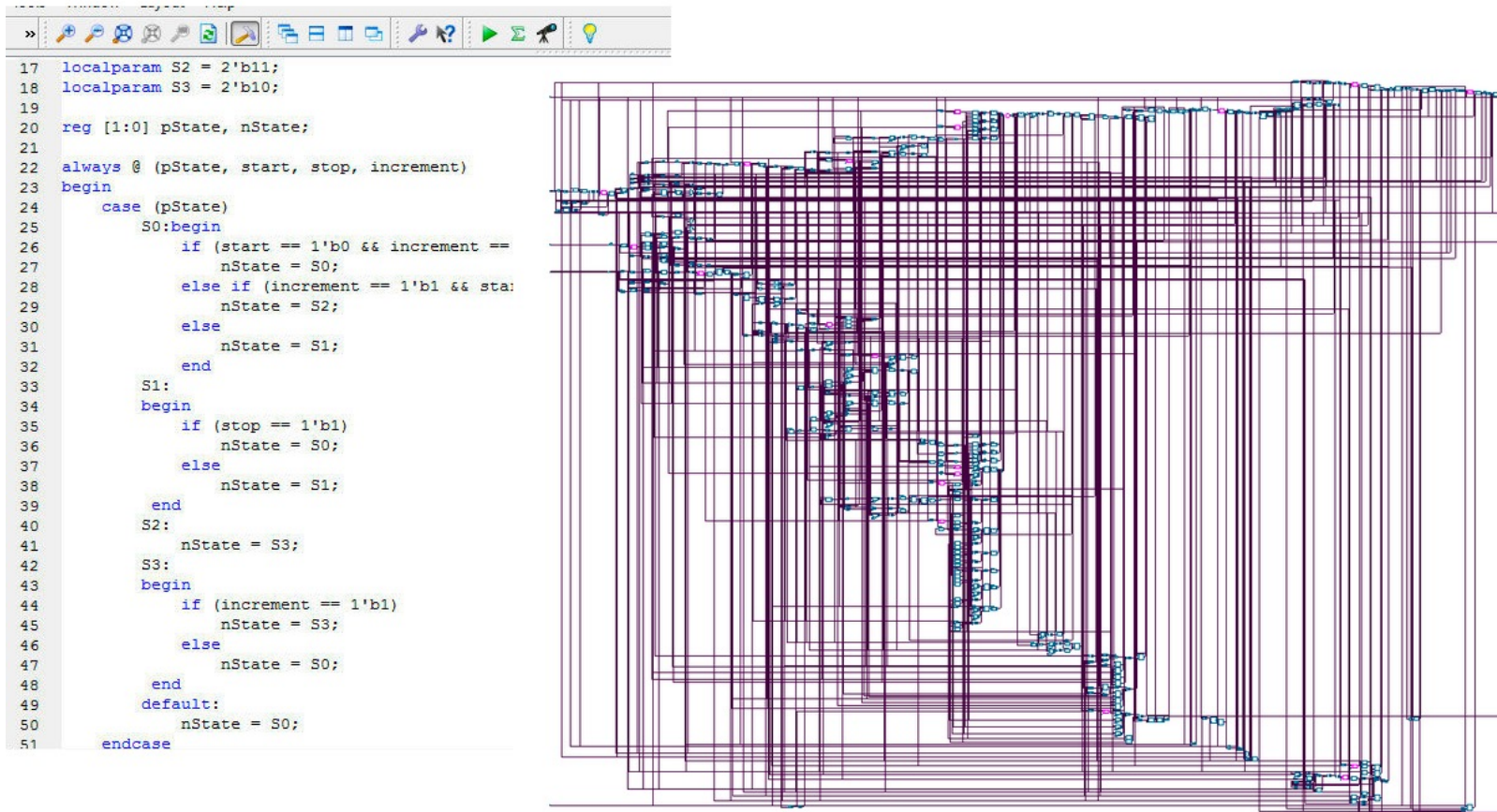


Hardware Description Languages (HDL)



Hardware Description Languages (HDL)

COMPLEX digital system (e.g. micro-processor) are designed and simulated writing code!
a **SYNTHESIS** tool generates the real **HARDWARE** for you !



Hardware Description Languages (HDL)

Hardware Description Languages (HDLs) :

- they are NOT "programming" languages !
- provide constructs for both implementation (describe the functionality of digital circuits) and verification (simulations, assertions etc.)
- synthesis (first step of modern digital design
- "synthesizable" constructs are only a minimal subset of the full language

Two main options on the market :

- VHDL
- Verilog

advanced options :

- high-level synthesis (HLS) using C/SystemC
- SystemVerilog
- analog and mixed-signal simulations (Verilog-A, Verilog-AMS, VHDL-AMS)

VHDL

- VHDL = VHSIC-HDL (Hardware Description Language)
- VHSIC = Very High Speed Integrated Circuit
- born in 1983 as a project from US Department of Defense (DoD)
- syntax derived from the **ADA programming language** (similar to PASCAL) as requested by DoD
- initially foreseen as a language used to describe and simulate digital integrated circuits
- later used also for digital synthesis
- 1987: first standardization as **IEEE Std. 1076-1987** (aka "VHDL 87")
- 1993: **first major revision** of the language as **IEEE Std. 1076-1993** (aka "VHDL 93")
- 2008: **second major revision** of the language as **IEEE Std. 1076-2008** (aka "VHDL 2008")
- provides constructs for both physical implementation (synthesizable) and simulation
- very reach syntax
- extremely **verbose** and **strongly typed** !

Verilog

- created by P. Goel, P. Moorby, C.L. Huang and D. Warmke between late 1983 and early 1984
- syntax derived from the **C programming language**
- introduced by Gateway Design Automation, later purchased by Cadence Design Systems in 1990
- born as a **verification** language for logic designs, intended to describe and simulate digital integrated circuits similar to VHDL
- initially a **proprietary** and **closed** language owned by Cadence
- later released by Cadence as an open language in order to cope with the increasing popularity of VHDL (already standardized by IEEE in 1987)
- 1995: first standardization by IEEE as **IEEE Std. 1364-1995** (aka "Verilog 95")
- 2001: first major revision of the language with extensions and new language features known as **IEEE Std. 1364-2001** (aka "Verilog 2001")
- 2006: second major revision with minor changes as **IEEE Std. 1364-2006** (aka "Verilog 2006")
- finally merged as a sub-set of the **SystemVerilog** HDL as **IEEE Std. 1800-2009**

Verilog vs. VHDL

Verilog and VHDL are the two most widespread HDLs in the world :

- approx. 50% market each one
- Verilog more popular in US and Japan, VHDL in Europe
- Verilog more used (and integrated with professional CAD tools) to design **Application-Specific Integrated Circuits** (ASICs)
- traditionally VHDL more used for **FPGA programming** instead
- both **equally** and well **supported** by **Xilinx**
- exception: gate-level timing simulations only supports Verilog netlists
- individual preference, usually mostly historical (that is, first language learned...)
- Verilog is **easier to learn** (C-like syntax), but also potentially more error-prone due to its relaxed data typing
- that is... you can make real disasters on silicon with Verilog, while using VHDL is pretty impossible
- starting alone from scratch... learn VHDL first, then move to Verilog once really annoyed with VHDL "verbose" coding

Digital simulators

Many different digital simulators available on the market.

From the "big three" :

- QuestaSim (the professional version of ModelSim) by Mentor (now Siemens)
- Xcelium (legacy Incisive) by Cadence
- VCS by Synopsys

Integrated with FPGA programming suites :

- XSim as part of the Vivado design suite by Xilinx
- ISim as part of the legacy ISE design suite by Xilinx
- ModelSim Altera as part of the Quartus design suite by Altera (now Intel)

Other solutions :

- Aldec HDL by Aldec
- open source simulators (FreeHDL, Icarus, GHDL etc,)

VHDL fundamentals

Case sensitivity, statements, white spaces

VHDL is **case insensitive** (on the contrary, Verilog is case sensitive) :

```
Z <= A and B ;
```

```
z <= a AND b ;
```

every VHDL statement is terminated with a semicolon ;

VHDL is also not sensitive to blanks (white spaces) between statements and empty lines are ignored by the compiler

Comments

in VHDL comments starts with two dashes :

```
-- this is a single-line VHDL comment  
  
-- this is a multiple-lines  
-- VHDL  
-- comment
```

VHDL lacks the possibility to quickly comment large sections of code (on the contrary, Verilog provides C-style block-comments enclosed within / and */)*

always put a lot of appropriate comments to document your code !

VHDL core elements

libraries and packages

- *import all useful data types and functions*

ENTITY declaration

- *interface of the digital block*
- *optional parameters*

ARCHITECTURE declaration

- *actual implementation of the block*

```
library ieee ;
use ieee.std_logic_1164.all ;

entity Inverter is
    port (
        X  : in  std_logic ;
        ZN : out std_logic
    ) ;
end entity Inverter ;

architecture rtl of Inverter is

begin

    ZN <= not X ;

end architecture rtl ;
```

Entity declaration

Any **digital block** implementing some functionality in VHDL is called *entity* :



the VHDL entity has I/O ports that can be declared as :

- *in*
- *out*
- *inout*
- *buffer*

optionally, the block can also include parameters (generics)

```
entity BlockName is

    -- optional generic list
    [generic( ... ) ; ]

    -- port list
    port( ... ) ;

end [entity] BlockName ;
```

Port list

Input ports :

```
clk : in std_logic ,  
rst : in std_logic , ...
```

Output ports :

```
busy : out std_logic ,  
LED   : out std_logic_vector (3 downto 0)
```

Inout (bidirectional) ports :

```
sda : inout std_logic
```

Architecture declaration

The actual block implementation is coded in the [architecture](#) instead :

```
architecture archName of BlockName is

    -- initial declarations
    ...

begin

    -- actual block implementation
    ...
    ...

end [architecture] archName ;
```

Testbench

In order to **simulate** the functionality of the digital block we also need a **testbench** that generates **stimuli** fed to input ports of our **Device Under Test (DUT)**:

```
entity tb_BlockName is
end entity ;

architecture testbench of tb_BlockName is
    ...
    ...
begin
    ...

    -- Device Under Test (DUT)
    DUT : BlockName port map (....) ;
    ...
end architecture ;
```

The module under test is always **instantiated** inside the testbench module, which contains **non-synthesizable code**.

VHDL data types

Logic constants

Fundamental logic constants in VHDL are '1' and '0':

single quotes for 1-bit (scalar) logic constants

```
'1'    '0'
```

double quotes for multiple-bits (buses) logic values

```
"0010"    "1010"
```

Built-in logic values

original VHDL-87 logic values :

- *bit* for 1-bit (scalar) signals
- *bit_vector* for multiple-bits (buses) signals

```
signal clock    : bit ;  
signal oneByte  : bit_vector(7 downto 0) ;
```

unresolved data types.....

not enough to describe actual digital circuits ! (e.g. high-impedance for 3-state buffers)

Extended logic values

extended VHDL logic values :

- *std_logic* for single-bit signals
- *std_logic_vector* for multiple-bits signals (buses, see later)

require external library as defined by IEEE Std. 1164

```
-- include resolved types
library IEEE ;
use IEEE.std_logic_1164.all ;

...

signal clock      : std_logic ;
signal oneByte    : std_logic_vector(7 downto 0) ;
```

- 1 logic one
- 0 logic zero
- Z high-impedance
- U uninitialized (sim. only)
- X unknown (driven)
- - don't care
- H weak high
- L weak low
- W weak signal

Buses and endianness

Most Significant Bit (MSB) vs. Least Significant Bit (LSB)
*the ordering of bits within a multi-bit string is referred to as **endianness***
big endian :

```
signal someSignal : std_logic_vector(11 downto 0)
```

little endian :

```
signal someSignal : std_logic_vector(0 to 4)
```

Sum between vectors (counting...)

In order to use the + operator between *std_logic_vector* you must use specific libraries:

- *IEEE.std_logic_unsigned*
- *IEEE.numeric_std*

```
library IEEE ;  
use IEEE.std_logic_1164.all ;  
use IEEE.std_logic_unsigned.all ;
```

IEEE.std_logic_unsigned is now DEPRECATED since:

- despite the prefix IEEE they are NOT IEEE standard, but **Synopsys proprietary** !
- since 1995, IEEE defines **numeric types** and **arithmetic functions** in the standard package *numeric_std* to outline a **true standard approach** for use arithmetic operator with synthesis tools

Signed/unsigned vectors

Extension of *std_logic_vector* :

- *signed*
- *unsigned*

```
library IEEE ;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all ;

...

signal count_s : signed(7 downto 0) := (others => '0') ;
signal count_u : unsigned(7 downto 0) := (others => '0') ;
```

The *IEEE.numeric_std* provides also provides functions to perform type-casting :

<= std_logic_vector(<unsigned signal>)

<= std_logic_vector(<signed signal>)

Binary codes

- *"straight" binary (the usual "power of 2")*
- *Gray code*
- *thermometer*
- *one-hot / one-cold*

Many many others : e.g. Hamming (for SEU protection)

Straight binary code

Usual binary strings interpreted as usual as "power of 2" integer numbers :

"000"	--	0
"001"	--	1
"010"	--	2
"011"	--	3
"100"	--	4
"101"	--	5
"110"	--	6
"111"	--	7

$$x = b_0 \times 2^0 + b_1 \times 2^1 + b_2 \times 2^2 + \dots + b_{N-1} 2^{N-1} = \sum_{i=0}^{N-1} b_i 2^i$$

Gray code

Only one bit change from a code to the next one :

"000"	--	0
"001"	--	1
"011"	--	2
"010"	--	3
"110"	--	4
"111"	--	5
"101"	--	6
"100"	--	7

Thermometer code

Continue to "padd" the code with a 1 on the right :

```
"0000000"  -- 0
"0000001"  -- 1
"0000011"  -- 2
"0000111"  -- 3
"0001111"  -- 4
"0011111"  -- 5
"0111111"  -- 6
"1111111"  -- 7
```

One-hot code

Only one bit set to one, then moving to the left :

"0000000"	--	0
"0000001"	--	1
"0000010"	--	2
"0000100"	--	3
"0001000"	--	4
"0010000"	--	5
"0100000"	--	6
"1000000"	--	7

The complementary is sometimes called "one cold"

Integer numbers

VHDL provides 32-bit integer numbers, can be signed or unsigned :

- *integer*
- *natural*
- *positive*

```
entity Counter is

    generic (
        N : natural := 2
    ) ;

    port (
        clk    : in  std_logic ;
        count  : out std_logic_vector(N-1 downto 0)
    ) ;

end entity Counter ;
```

Real

real 64-bit double-precision real numbers :

Example :

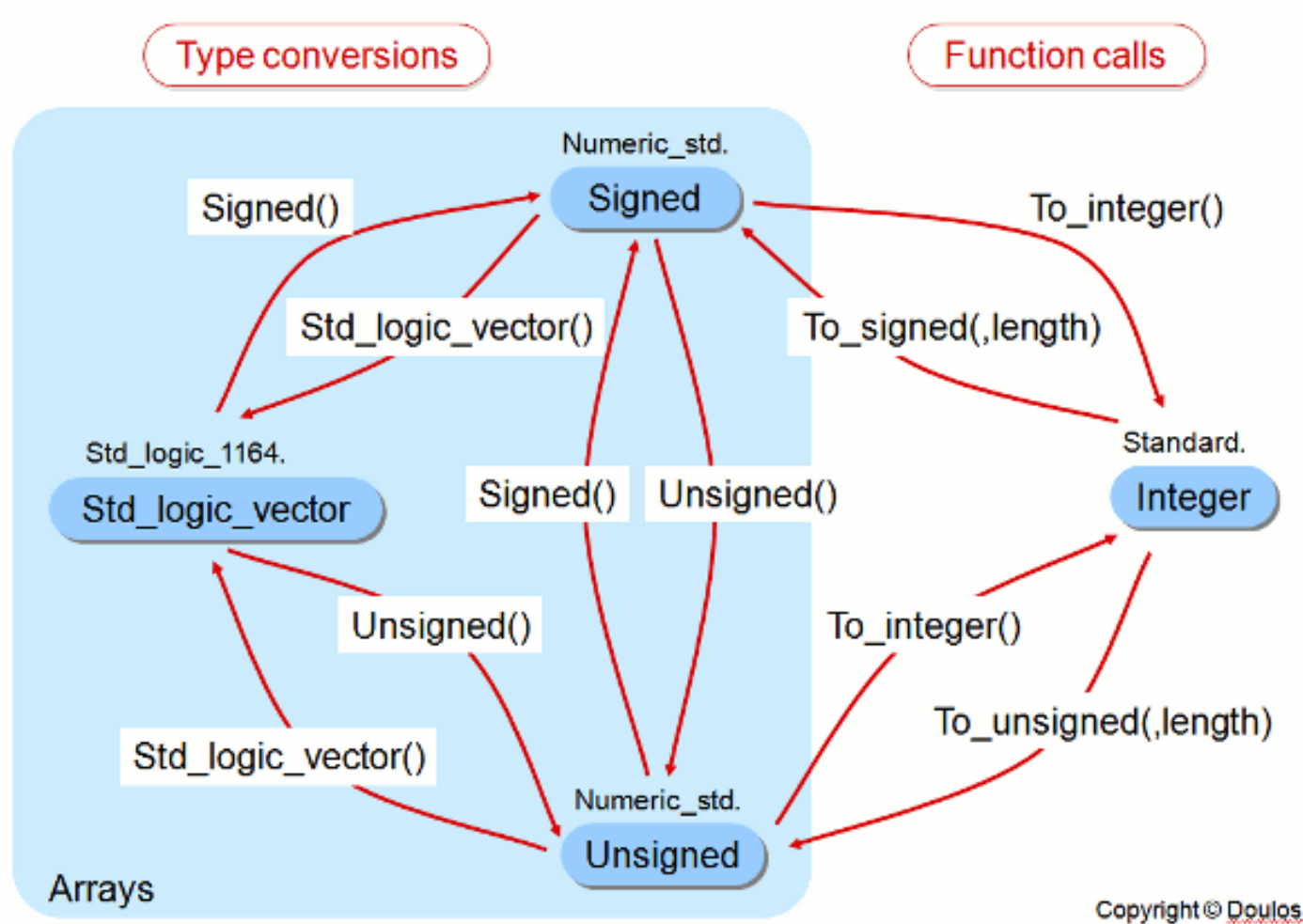
Useful for Real-Number Modeling (RNM)

```
constant VDD : real := 1.2 ;
```

Physical data types

```
constant PERIOD : time := 10 ns ;  
signal clk : std_logic ;  
  
...  
  
clock : process  
begin  
    clk <= '0' ;  
    wait for PERIOD/2 ;  
    clk <= '1' ;  
    wait for PERIOD/2 ;  
end process ;
```

Numeric Std Conversions



Composite type

A composite type is a datatype consisting of several subelements.

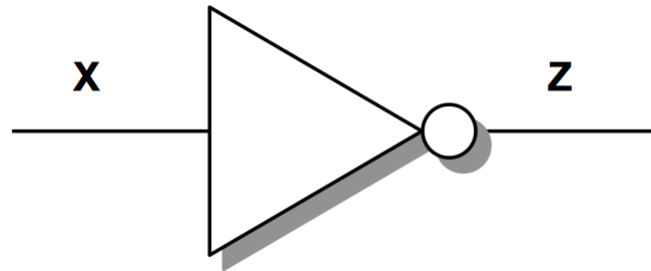
```
1  type arr_type is array (0 to 255) of real;
2
3  type rec_type is record
4      int_element : integer;
5      slv_element : std_logic;
6  end record;
7
8  signal my_arr : arr_type;
9  signal my_rec : rec_type;
10 signal my_slv : std_logic_vector(31 downto 0);
```

```
1 type example_fsm_t is (idle, starting, running, stopping);
```

```
1 -- Declare a signal which uses our custom type
2 signal example : example_fsm_t;
3 -- Example of assigning a value to our signal
4 example <= stopping;
```

Boolean algebra

NOT gate

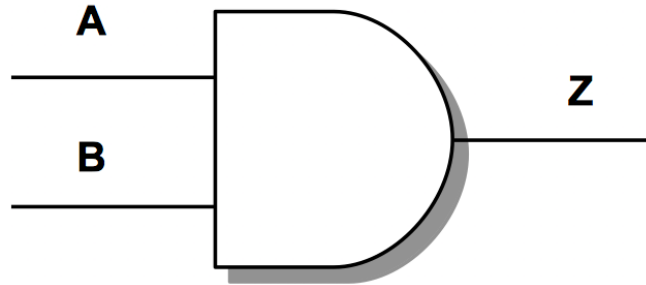


$$z = \bar{X}$$

VHDL syntax :

```
Z <= not X ;
```

AND gate

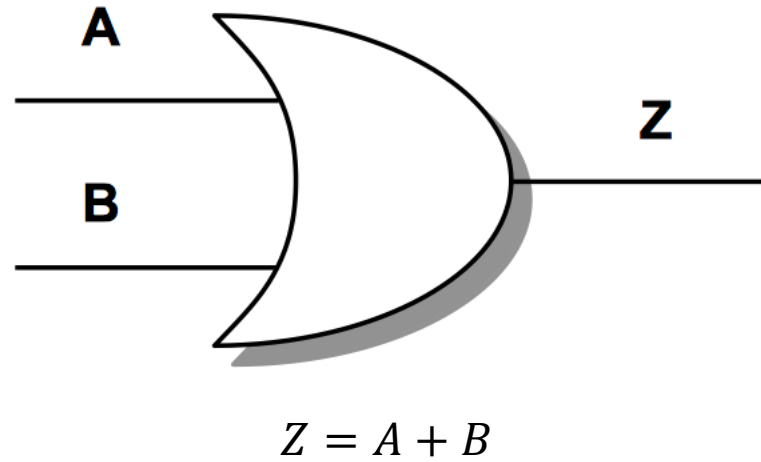


$$Z = A \cdot B$$

VHDL syntax :

```
Z <= A and B ;
```

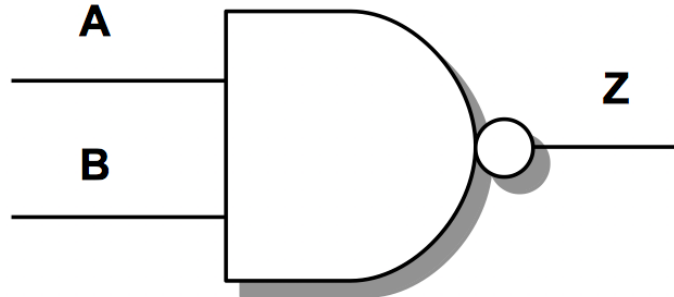
OR gate



VHDL syntax :

```
Z <= A or B ;
```

NAND gate

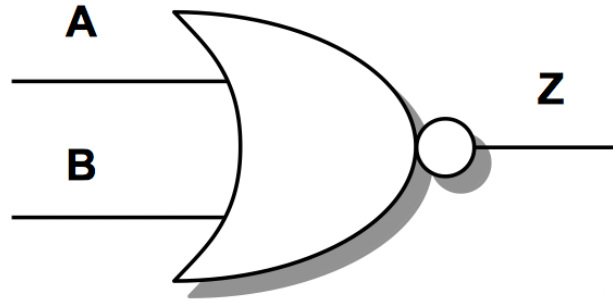


$$Z = \overline{A \cdot B}$$

VHDL syntax :

```
Z <= A nand B ;
```

NOR gate



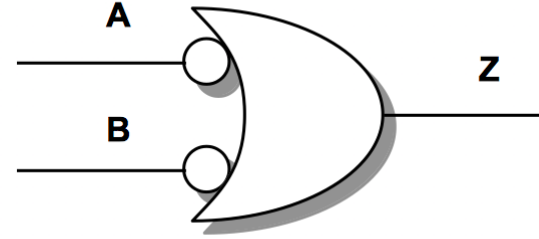
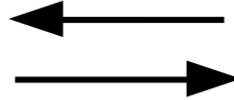
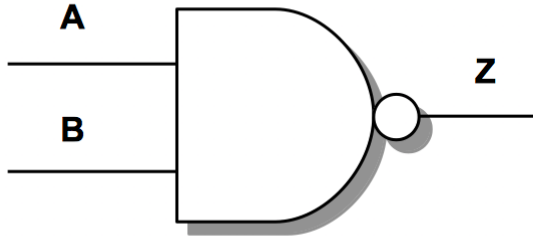
$$Z = \overline{A + B}$$

VHDL syntax :

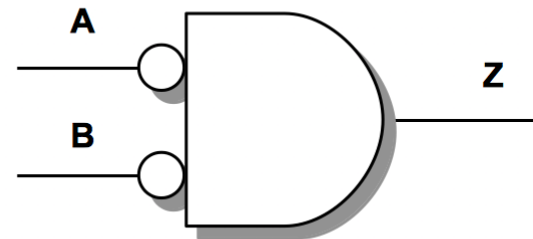
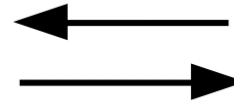
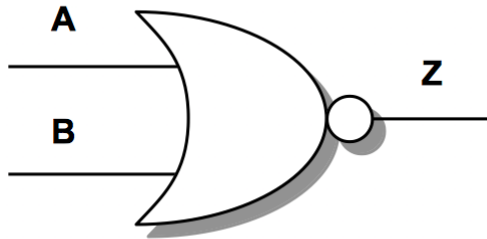
```
Z <= A nor B ;
```

De Morgan's theorem

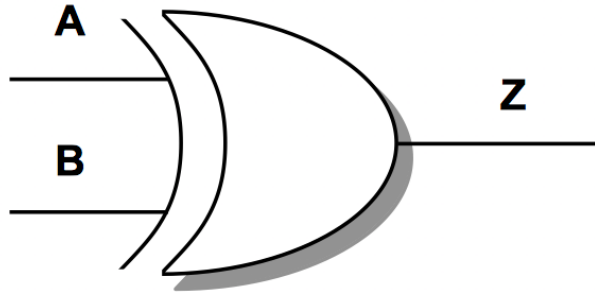
$$\overline{A \cdot B} = \overline{A} + \overline{B}$$



$$\overline{A + B} = \overline{A} \cdot \overline{B}$$



XOR gate

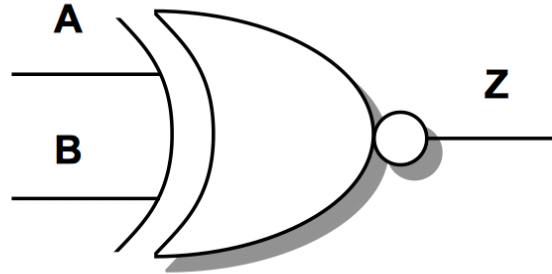


$$Z = A \oplus B = (A \cdot \overline{B}) + (\overline{A} \cdot B)$$

VHDL syntax :

```
Z <= A xor B ;
```

XNOR gate



$$Z = \overline{A \oplus B} = (A \cdot B) + (\overline{A} \cdot \overline{B})$$

VHDL syntax :

```
Z <= A xnor B ;
```

- *not implemented in the original VHDL-87*
- *later added as part of VHDL-93 revision upon designers request*
- *detects if two bits are identical*
- *fundamental component for binary comparators*

Propagation delays

VHDL syntax :

```
SUM    <= A xor B after 5 ns ;  
CARRY <= A and B after 3 ns ;
```

Only for simulation purposes !

Multiple-inputs gates

```
Z <= X(0) and X(1) and X(2) and ... and X(N-1) ;
```

```
Z <= X(0) or X(1) or X(2) or ... or X(N-1) ;
```

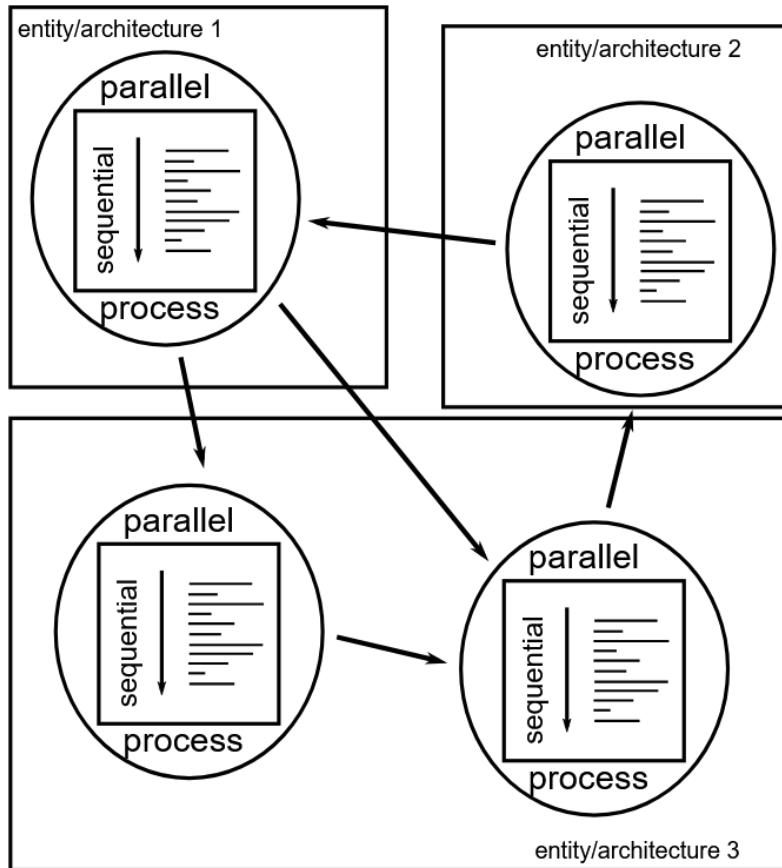
```
Z <= X(0) nand X(1) nand X(2) nand ... nand X(N-1) ;  
WRONG !
```

```
Z <= not( X(0) and X(1) and X(2) and ... and X(N-1)) ;
```

when/else conditional signal assignment

```
b <= "1000" when a = "00" else  
    "0100" when a = "01" else  
    "0010" when a = "10" else  
    "0001" when a = "11" ;
```

VHDL 'process' structure



“In VHDL, the process statement contains **sequential** statements. Processes are only permitted inside an architecture. The statements within processes execute sequentially, not concurrently.”

Combinational logic

```
1  MUX_2_1 : process(sel, in_a, in_b)
2  begin
3      case sel is
4          when '0' =>
5              out_q <= in_a;
6          when '1' =>
7              out_q <= in_b;
8          when others =>
9              out_q <= 'X';
10     end case;
11 end process;
```

Sequential logic

```
1  DELAY_PROC : process(clk)
2  begin
3      if rising_edge(clk) then
4          if rst = '1' then
5              sig_p1 <= '0';
6
7          else
8              sig_p1 <= sig;
9
10         end if;
11     end if;
12 end process;
```

If/else statement

```
31
32 entity XuLA_2 is
33     Port ( PB1      : in  STD_LOGIC;
34            PB2      : in  STD_LOGIC;
35            PB3      : in  STD_LOGIC;
36            PB4      : in  STD_LOGIC;
37            LED1     : out  STD_LOGIC;
38            LED2     : out  STD_LOGIC;
39            LED3     : out  STD_LOGIC;
40            LED4     : out  STD_LOGIC);
41 end XuLA_2;
42
43 architecture Behavioral of XuLA_2 is
44 begin
45
46     process (PB1)
47     begin
48         if PB1 = '1' then
49             LED1 <= '1';
50             LED2 <= '0';
51             LED3 <= '1';
52             LED4 <= '0';
53         else
54             LED1 <= '0';
55             LED2 <= '1';
56             LED3 <= '0';
57             LED4 <= '1';
58         end if;
59     end process;
60
61 end Behavioral;
```

Case/when statement

```
31
32 entity XuLA_2 is
33     Port ( PB1      : in  STD_LOGIC;
34            PB2      : in  STD_LOGIC;
35            PB3      : in  STD_LOGIC;
36            PB4      : in  STD_LOGIC;
37            LED1     : out  STD_LOGIC;
38            LED2     : out  STD_LOGIC;
39            LED3     : out  STD_LOGIC;
40            LED4     : out  STD_LOGIC);
41 end XuLA_2;
42
43 architecture Behavioral of XuLA_2 is
44 begin
45
46     process (PB1)
47     begin
48         case PB1 is
49             when '1' => LED1 <= '1';
50                        LED2 <= '0';
51                        LED3 <= '1';
52                        LED4 <= '0';
53             when others => LED1 <= '0';
54                        LED2 <= '1';
55                        LED3 <= '0';
56                        LED4 <= '1';
57         end case;
58     end process;
59
60 end Behavioral;
61
```